



Access Entries and Access Policies

The principal that creates a cluster is granted cluster administrator on it. Nobody else starts with anything: a teammate, a CI pipeline, or a second role you assume yourself is refused at the API server until a grant is written for it.

IN THIS LECTURE

01 Two objects build one grant

The entry names the principal, the associations carry the permissions.

02 Grants combine by union

Every association and every group name adds together, and nothing subtracts.

03 Seven published policies

You cannot edit them, and two of the names understate what they grant.

04 Scope decides where a policy applies

Cluster scope covers every namespace, namespace scope the names you list.

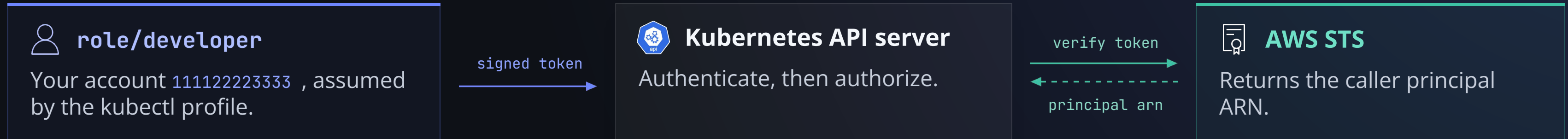


TWO OBJECTS, ONE GRANT

The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING

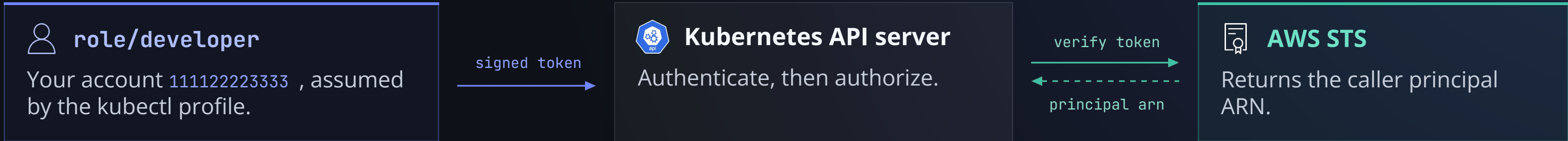


TWO OBJECTS, ONE GRANT

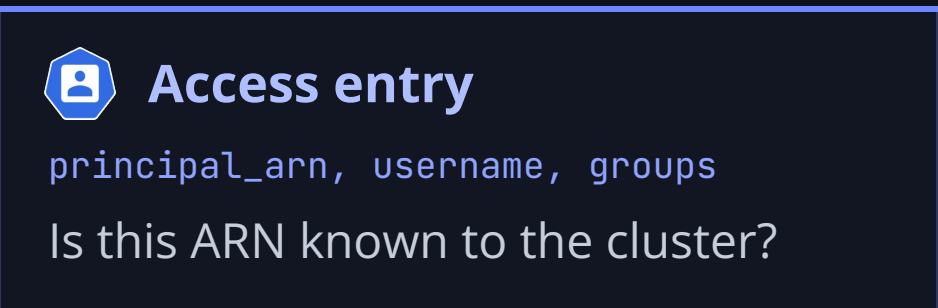
The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING



AUTHORIZATION · EKS CONTROL PLANE, CLUSTER eks-access-management

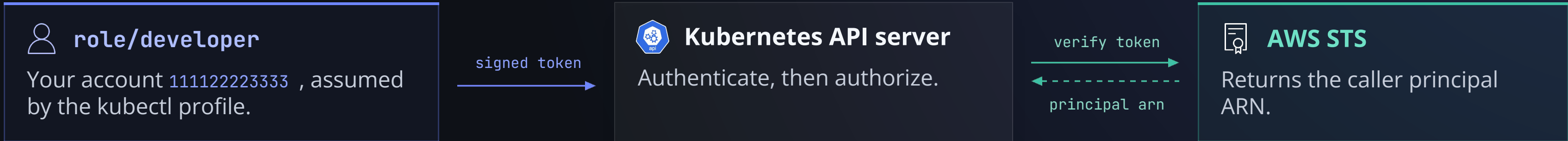


TWO OBJECTS, ONE GRANT

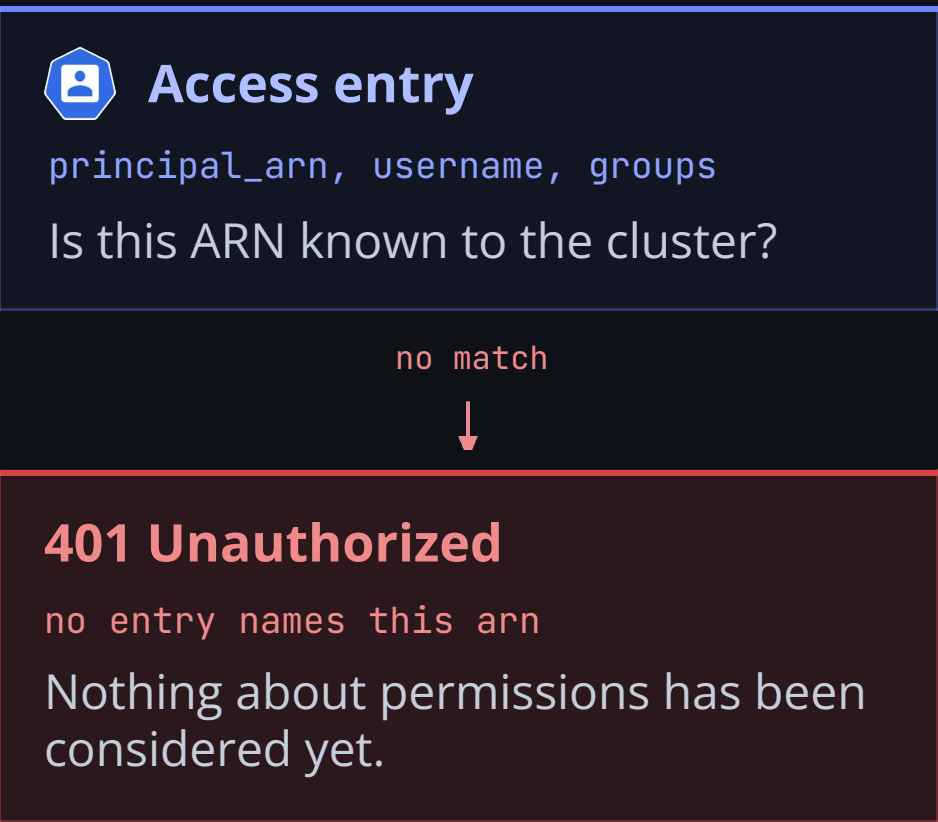
The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING



AUTHORIZATION · EKS CONTROL PLANE, CLUSTER eks-access-management

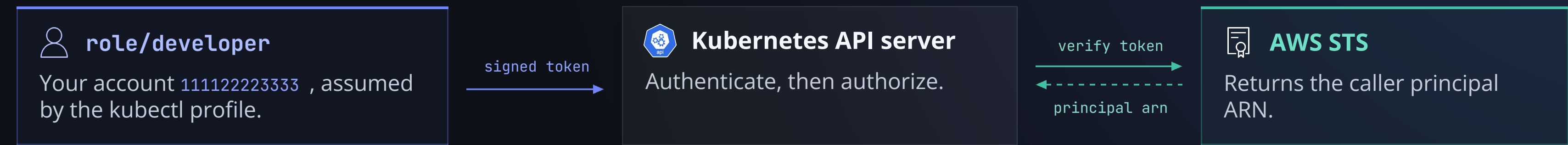


TWO OBJECTS, ONE GRANT

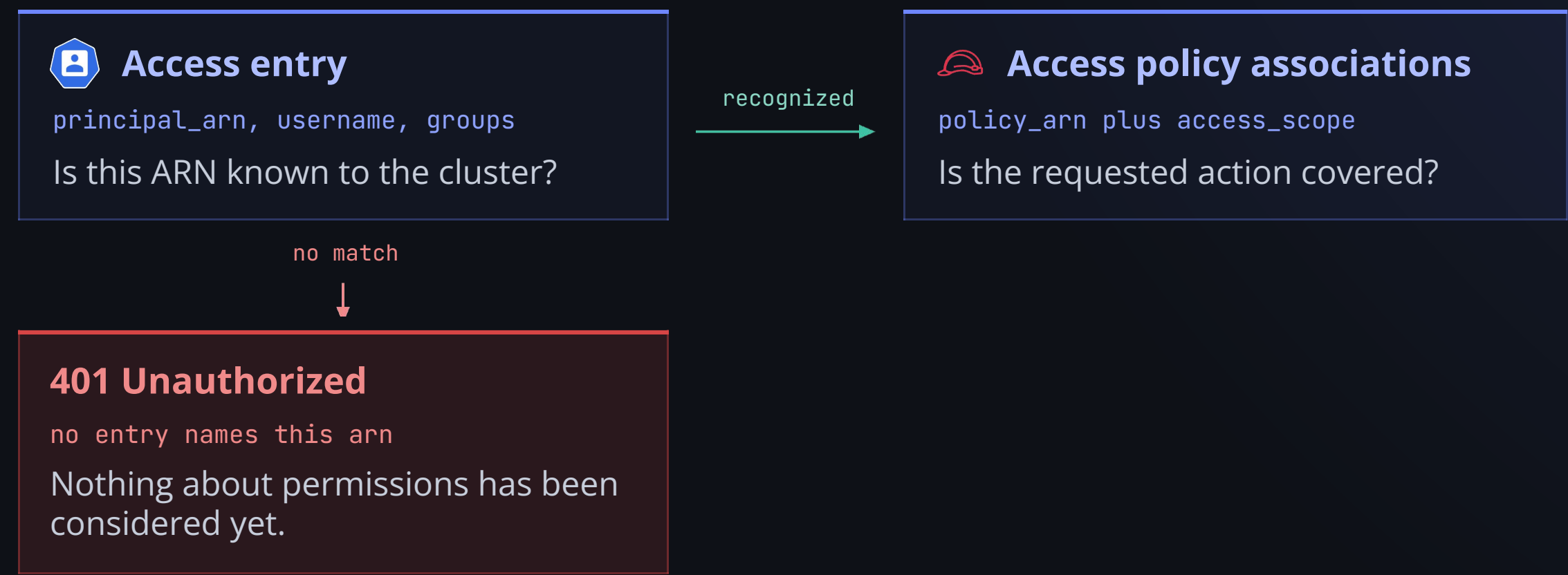
The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING



AUTHORIZATION · EKS CONTROL PLANE, CLUSTER eks-access-management

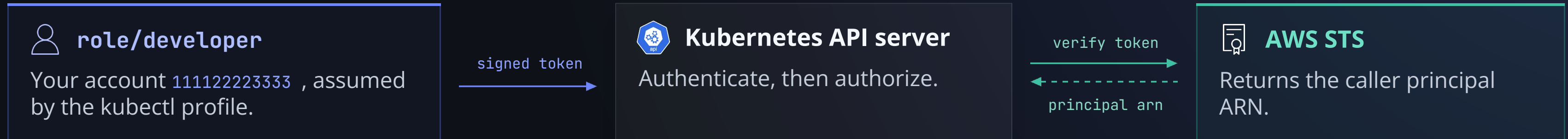


TWO OBJECTS, ONE GRANT

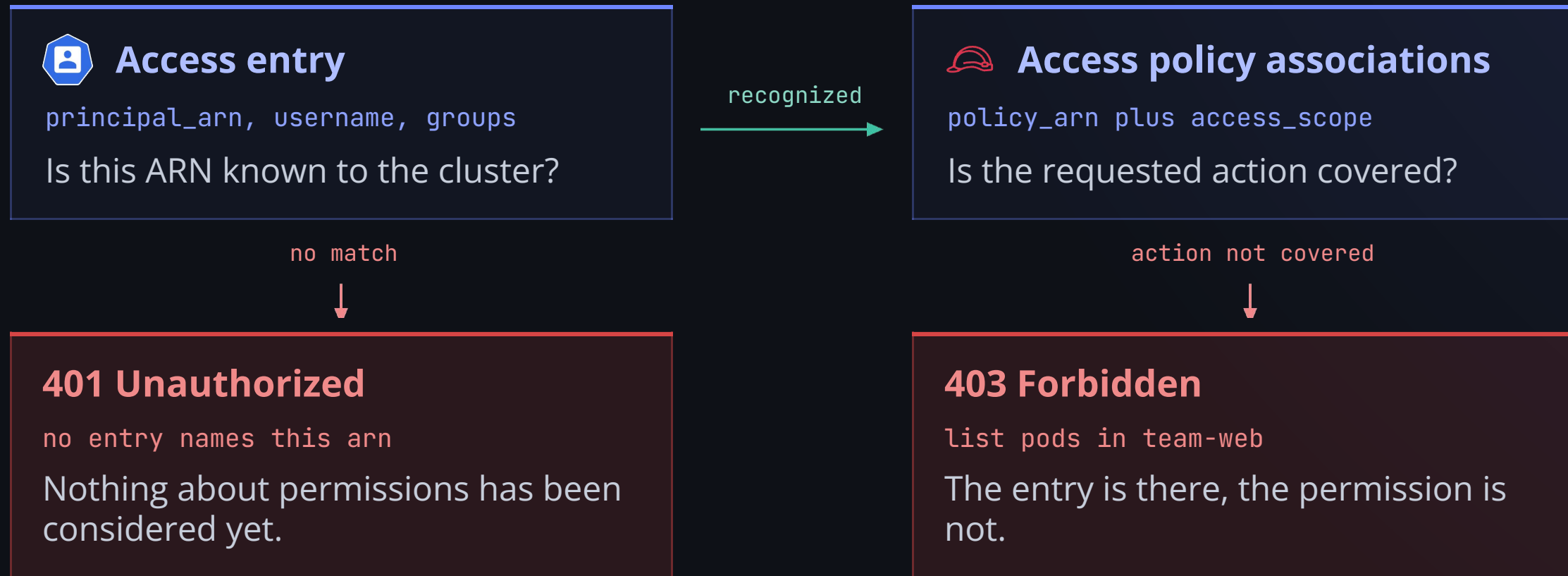
The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING



AUTHORIZATION · EKS CONTROL PLANE, CLUSTER eks-access-management

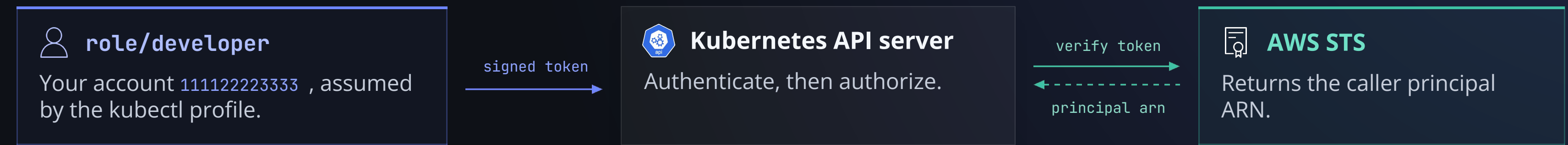


TWO OBJECTS, ONE GRANT

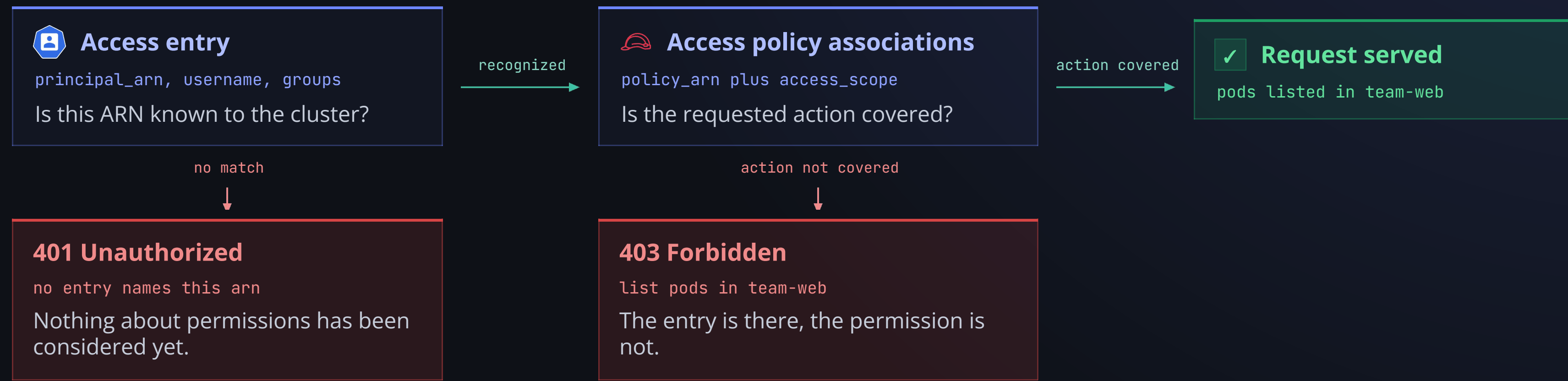
The entry answers who, the associations answer what

A principal with no entry is refused with 401. A recognized principal asking for an action no association covers is refused with 403.

AUTHENTICATION · WHO IS CALLING



AUTHORIZATION · EKS CONTROL PLANE, CLUSTER eks-access-management



Two resources joined only by the principal ARN

The association carries no entry ID field, so the reference you write in `principal_arn` is what decides the creation order.

access.tf

```
resource "aws_eks_access_entry" "developer" {
  cluster_name = "eks-access-management"
  principal_arn = aws_iam_role.developer.arn
  type         = "STANDARD"
}

resource "aws_eks_access_policy_association" "developer_view" {
  cluster_name = "eks-access-management"
  principal_arn = aws_eks_access_entry.developer.principal_arn
  policy_arn    = "arn:aws:eks::aws:cluster-access-policy/AmazonEKSVIEWPolicy"
  access_scope {
    type = "cluster"
  }
}
```

`principal_arn`

The association names no entry ID

Both resources carry the same principal ARN, and EKS matches them on that value alone.

`aws_eks_access_entry.developer.principal_arn`

Referencing the entry gives Terraform the dependency edge

Writing the role ARN as a literal in both places lets Terraform attempt the association first, which fails.

`type`

The type is fixed at creation

STANDARD is the default and the only type that accepts a policy association or a Kubernetes group name.

Seven general-purpose policies, and none of them editable

Read the Secrets column rather than the name. View is the only one of the seven that cannot reach a Secret.

Policy	What it grants	Secrets
AmazonEKSVIEWPolicy	get, list, watch on most namespaced resources	× excluded
AmazonEKSEditPolicy	View plus create, update, delete, and pods/exec	read and write
AmazonEKSAdminPolicy	Edit plus Roles and RoleBindings	read and write
AmazonEKSClusterAdminPolicy	every API group, every resource, every verb	every verb
AmazonEKSAdminViewPolicy	get, list, watch on every resource type	read
AmazonEKSSecretReaderPolicy	get, list, watch on Secrets alone	read
AmazonEKSSecretAdminPolicy	every verb on Secrets alone	every verb



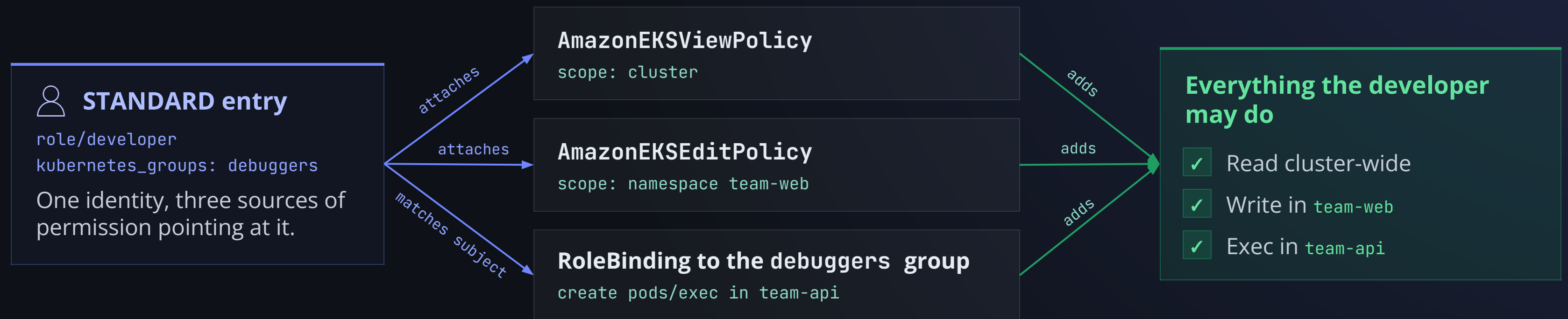
Two names understate what they grant
Edit reads and writes Secrets and opens a shell in any pod in its namespaces. View sees no Secret at all.



The rest of the published list is not here
Those policies exist for Auto Mode, hybrid nodes, backup and the capability controllers rather than for a person or a pipeline.

Every grant on one identity is added together

There is no deny statement in this model, no precedence between associations, and no most-specific-wins rule.



! Adding a narrower grant removes nothing

A principal holding ClusterAdmin who is then given View scoped to one namespace is still a full cluster administrator. Access narrows only by editing or deleting the grant that is already there.

Access scopes limit the extent of permissions

Scope sits on the association rather than on the entry, so one principal can hold the same policy at two different reaches.



Cluster scope

type = "cluster"

```
access_scope {  
  type = "cluster"  
}
```

- ✓ The policy applies across the whole cluster in one association.
- ! Namespaces created after the grant was written fall inside it with no edit.
- ✓ The block carries no `namespaces` list, so there is nothing to keep in step as the cluster grows.



Namespace scope

type = "namespace"

```
access_scope {  
  type      = "namespace"  
  namespaces = ["team-web", "team-*"]  
}
```

- ✓ One list holds several names, and `team-*` matches every namespace whose name starts with `team-`.
- ✓ Names can be added or removed, and the scope switched to cluster, after the association exists.



EKS does not check that a namespace named in a scope exists

A misspelled name such as `tem-web` is accepted, applies cleanly, and grants nothing. The principal is refused every action while the association reads as correct in the console.